Improved Methodology & Implementation Guide

Transparent Policy GraphRAG

A Supervisor-Friendly Reconstruction & Research Positioning Guide

---

Author: Alireza (Allen) Sharafzad

MSc Data Science & AI, Bournemouth University

Supervisor: Dr Sofia Meacham

Date: 11 June 2026

Purpose of this document

This guide explains, in plain terms, how the Transparent Policy GraphRAG system is built, and provides the ordered sequence of prompts needed to reconstruct it with an AI coding agent (Claude). Each prompt can be copied and pasted as-is; a short check after each one confirms the step succeeded. No prior knowledge of knowledge graphs or Neo4j is assumed.

1. Introduction

1.1 What the system does

Transparent Policy GraphRAG answers natural-language questions about university policy documents — for example, *"What is the thesis word limit for a Law student?"* — and, crucially, **shows its working**. Every answer comes with the exact policy section it is based on, a visual map of the rules involved, and a record of how the answer was found.

It does this in three stages:

1. **Read** — a policy PDF is converted into a strictly structured XML format where every rule keeps a reference to its original section number.

2. **Map** — that XML is loaded into a *knowledge graph*: a database where policies, rules, conditions and outcomes are stored as connected nodes rather than loose text. (Think of it as a mind-map the computer can query precisely.)

3. **Answer** — questions are answered by querying the graph first, with intelligent fallbacks, and every retrieved context is *validated* before an answer is written. If the system cannot find relevant material, it says so instead of guessing.

1.2 How to use this guide

- Work through the prompts **one at a time, in order**, in a single Claude session per phase.
- Copy each grey block exactly as printed.
- After each prompt, confirm the **Check** line is satisfied before moving on.
- The phases build on each other: 1 → 2 → 3 → 4 → Final Integration.
- Prompts marked **[KEY]** carry the central architectural ideas.

1.3 What you need

Requirement	Details
Python 3.11	with Streamlit 1.35+
Neo4j AuraDB	free cloud tier; credentials kept in a local `.env` file
OpenAI API key	GPT-4o, GPT-4o-mini, and the `text-embedding-3-small` embedder
A policy document	e.g. the BU Code of Practice for Research Degrees 2024–25 (PDF)

2. Phase 1 — From PDF to Validated XML

Goal: turn an unstructured policy PDF into clean, structured XML in which every element is traceable to a real section of the document, and meaningless placeholder entries (e.g. "Untitled") are impossible.

Prompt 1.1 — Extract the PDF text (fail loudly)

Copy and paste this block to Claude:

```
Implement _extract_pdf_text(pdf_bytes, log_fn) using PyMuPDF. Read every
page and return the combined text. If the result is empty (a scanned,
image-only PDF), the caller process_pdf_to_xml must raise
RuntimeError("No extractable text found in the PDF.") rather than
continuing silently. Send all status messages through log_fn(level, message).
```

Check: a normal PDF returns text; a scanned image PDF raises a visible error.

Prompt 1.2 — Split the text into overlapping chunks

Copy and paste this block to Claude:

```
Implement _chunk_text(text, chunk_size, overlap) and call it with
CHUNK_SIZE = 15000 and CHUNK_OVERLAP = 1500 characters. The overlap
ensures a section heading cut at a chunk boundary still appears whole
in at least one chunk. Log the character count and number of chunks.
```

Check: a 60,000-character document yields about 5 overlapping chunks.

Prompt 1.3 — Write the extraction rules (the "no placeholders" contract) [KEY PROMPT]

Copy and paste this block to Claude:

```
Write the PDF_TO_XML_SYSTEM prompt for the extraction model. It must
require every element ID to be anchored to a real section number:

  Rule       R_{Section}              e.g. R_13, R_7_2
  Condition  C_{Section}_{Slug}       e.g. C_13_1_LawThesisWordLimit
  Outcome    O_{Section}_{Slug}       e.g. O_7_2_WithdrawalSanction

The Slug is a short PascalCase phrase taken ONLY from the actual policy
text. Forbid "Untitled", "Unknown", empty IDs, and bare IDs like R1.
Decisive rule: if a descriptive name cannot be derived from the text,
do not emit the element at all — omission is recoverable, a junk node
is not.
```

Check: sample output produces IDs like `C_13_1_LawThesisWordLimit`; never `Untitled` or `R1`.

Prompt 1.4 — Run extraction chunk by chunk

Copy and paste this block to Claude:

```
In process_pdf_to_xml, create one ChatOpenAI(model="gpt-4o",
temperature=0, max_tokens=8000) instance. For each chunk, send
PDF_TO_XML_SYSTEM plus: "CHUNK i OF N: Derive all element IDs from
section numbers visible in THIS chunk's text." Skip empty responses
with a warning; on an API error, log it and re-raise so the user
knows extraction failed. Raise if OPENAI_API_KEY is missing.
```

Check: N chunks produce up to N XML fragments; an API failure stops the run visibly.

Prompt 1.5 — Merge fragments, remove duplicates and junk

Copy and paste this block to Claude:

```
Implement _merge_xml_fragments(raw_fragments, log_fn):
- _sanitise: strip Markdown fences and XML prologs, escape stray "&",
  remove control characters.
- Parse each fragment; a single bad fragment is counted and skipped,
```

```
  never aborts the merge.
- Deduplicate by tag + id. If the same Rule appears in two overlapping
  chunks, merge their Conditions/Outcomes into one copy.
- _is_generic: reject any element whose id or label is "untitled",
  "unknown", "unnamed", "n/a", "none", empty, or matches ^[rco]\d{1,3}$.
  Count and report everything rejected.
```

Check: a Rule found in two chunks becomes one node with the union of its children; no junk IDs survive.

Prompt 1.6 — Final validation gate

Copy and paste this block to Claude:

```
Implement _wrap_xml_fragment to produce one <Policy>...</Policy>
document, then parse it with ElementTree as the last step before
returning. On a parse error, report the exact offending line and
re-raise — malformed XML must never reach the database.
```

Check: the function returns parseable XML; a corrupted fragment is reported with its line number.

3. Phase 2 — Building the Knowledge Graph

Goal: load the validated XML into Neo4j as a connected graph — each policy in its own namespace, every node given a human-readable label, risky clauses flagged, and text embeddings attached for semantic search.

Prompt 2.1 — Create the policy root node

Copy and paste this block to Claude:

```
Implement ingest_xml_to_neo4j(xml_content, graph, log_fn, policy_title,
policy_version). Build a namespace id: pid = slug(title) + "-" +
slug(version). Start with a clean-slate wipe (MATCH (n) DETACH DELETE n),
then MERGE one (:Policy {id: $pid}) node with title, version and an
ingestion timestamp. Document that re-ingesting replaces the whole graph.
```

Check: ingestion produces exactly one Policy node; any previous graph is cleared first.

Prompt 2.2 — Create the vector indices safely

Copy and paste this block to Claude:

```
Before creating member nodes, DROP and re-CREATE two vector indices
(rule_text_index on Rule, condition_text_index on Condition) using the
module-level EMBEDDING_DIMS (1536 normally; 384 if the offline fallback
embedder is active) with cosine similarity. This prevents a stale index
when the embedding dimension changes.
```

Check: both indices exist at the active dimension; switching embedders and re-ingesting does not error.

Prompt 2.3 — Load Rules, Conditions and Outcomes

Copy and paste this block to Claude:

```
For each <Rule> in the XML, MERGE a (:Rule {id, policy_id}) node with its
label, source section and description, and connect it to the Policy with
BELONGS_TO. MERGE each child Condition and Outcome the same way and
connect with HAS_CONDITION / HAS_OUTCOME. Keep a stats dictionary of
counts. The section-anchored id is the MERGE key, so re-ingestion is
idempotent and a clause shared by two rules becomes ONE node with two
edges.
```

Check: node counts match the XML; a condition referenced by two rules exists once with two `HAS_CONDITION` edges.

Prompt 2.4 — Generate friendly labels

Copy and paste this block to Claude:

```
Implement generate_human_label(text) with @st.cache_data(max_entries=2048)
so identical text is never billed twice. Use gpt-4o-mini (temperature 0,
max_tokens 20) to produce a 2-3 word title, capped at 4 words. If the
call fails, fall back to a 40-character truncation. Store as label_human;
keep the precise id and section for the audit trail.
```

Check: `C_13_1_LawThesisWordLimit` displays as something like "Thesis Word Limit"; repeats trigger no second API call.

Prompt 2.5 — Flag risks and attach embeddings

Copy and paste this block to Claude:

```
Implement is_risk_text(*parts) that matches RISK_KEYWORDS (withdraw,
sanction, penalty, failure, terminate, expel, revoke, reject, ...) and
set is_risk on matching nodes so they render red in the Reasoning View.
For each Rule and Condition, embed "human label · label · description"
```

```
with _embed_with_retry and store it on the node's embedding property.
```

Check: sanction-related nodes carry `is_risk = true`; vector queries against both indices return results.

Prompt 2.6 — Support conflict edges and richer relations

Copy and paste this block to Claude:

```
Allow deterministic CONFLICTS_WITH edges between Conditions, with
properties scenario, resolution, sections, cross_policy. Separately, in
the CLI prototype graphrag_policy_bot.py, write CYPHER_GENERATION_TEMPLATE
teaching the query model the fuller vocabulary (HAS_CONDITION, HAS_OUTCOME,
GOVERNED_BY, SATISFIES, TRIGGERS, ESCALATES_TO, OVERRIDES, CONFLICTS,
RESTRICTS, BELONGS_TO). Require every query to return the source section,
and to traverse OVERRIDES and CONFLICTS for eligibility questions.
```

Check: the schema accepts a fully-propertied conflict edge; the prototype generates multi-hop queries for escalation questions.

Prompt 2.7 — List and delete policies correctly

Copy and paste this block to Claude:

```
Implement get_ingested_policies(graph), projecting all aggregates through
an explicit WITH clause and ordering by aliases (newer Neo4j forbids using
pre-aggregation variables afterwards). Do not swallow errors — the sidebar
must show a real error message, never a silently empty list. Implement
delete_policy(graph, policy_id) that detaches and deletes one policy's
sub-graph without touching others.
```

Check: the sidebar lists the policy with correct counts; a query fault shows an explicit error.

4. Phase 3 — Asking Questions: Hybrid Search, Routing & Validation

Goal: a query pipeline that classifies each question, tries a precise structured query first, falls back to hybrid semantic + keyword search when needed, and **validates** the retrieved evidence before any answer is written.

Prompt 3.1 — Clean the query and choose a route [KEY PROMPT]

Copy and paste this block to Claude:

```
Implement _clean_user_query(raw) to strip whitespace and UI artefacts.
Implement route_query using gpt-4o (temperature 0, max_tokens 50) that
returns exactly:
    ROUTE: <DIRECT_LOOKUP|COMPLEX_REASONING>
    REASON: <one phrase, max 12 words>
DIRECT_LOOKUP = simple single-fact questions. COMPLEX_REASONING =
multi-step, comparative or thematic questions. FAIL-OPEN: any error or
missing key defaults to COMPLEX_REASONING (the most thorough path).
Set use_fusion = (route == "COMPLEX_REASONING") and pass it onward.
```

Check: "What is the draft submission deadline?" → DIRECT_LOOKUP; a comparison question → COMPLEX_REASONING; an outage still routes safely.

Prompt 3.2 — Try the structured query first [KEY PROMPT]

Copy and paste this block to Claude:

```
In run_query, always run the GraphCypherQAChain first: gpt-4o writes a
Cypher query from the live schema, Neo4j executes it, gpt-4o writes the
answer. Capture the generated Cypher and the raw rows for the audit
trail. Only engage the fallback when the result is empty or the answer
contains the sentinel "not available in the current policy graph".
```

Check: an answerable factual question never enters fallback; an out-of-scope question does.

Prompt 3.3 — Rephrase complex questions three ways

Copy and paste this block to Claude:

```
Implement generate_multiple_queries(user_query, num_queries=3) with
gpt-4o (temperature 0): one formal/regulatory phrasing, one plain
student-facing phrasing, one synonym/adjacent phrasing. Output the
questions only, one per line. On any failure, fall back to the original
query alone.
```

Check: a complex question yields up to 3 rephrasings plus the original; failure degrades gracefully.

Prompt 3.4 — Combine rankings fairly (Reciprocal Rank Fusion)

Copy and paste this block to Claude:

```
Implement reciprocal_rank_fusion(ranked_lists, k=60): each item scores
the sum of 1/(k + rank) across all lists it appears in, sorted
descending. Items ranked highly in several lists win.
```

Check: a node ranked #1 in two lists outranks a node ranked #1 in only one; output is deterministic.

Prompt 3.5 — The two-mode search engine [KEY PROMPT]

Copy and paste this block to Claude:

```
Implement _semantic_search(graph, question, top_k=8, threshold=0.70,
score_window=12, use_fusion=True).

COMPLEX mode (use_fusion=True): embed the original + rephrased queries,
search both vector indices for each, fuse the rankings with RRF (k=60),
and keep candidates whose best cosine similarity is >= 0.70.
```

```
DIRECT mode (use_fusion=False): run TWO streams — (A) vector search on
the single original query; (B) keyword search counting how many domain
keywords (word, limit, thesis, deadline, viva, penalty, ...) appear in
each node's text. Fuse A + B with RRF and keep the strict top 5 by rank
— do NOT apply the cosine threshold here, so a node found only by
keyword (cosine 0.0) still survives. Log every candidate's cosine and
RRF score with a kept/dropped flag.
```

Check: in direct mode a keyword-only hit survives; in complex mode only ≥ 0.70 cosine survives; the log shows per-node scores.

Prompt 3.6 — Wrap the fallback

Copy and paste this block to Claude:

```
Implement keyword_fallback_search(graph, question, use_fusion) that runs
_semantic_search first and only drops to a plain text-CONTAINS query if
vector search returns nothing. Return the annotated query, rows, scores.
```

Check: with embeddings present the vector path is used; without them, a plain query still returns candidates.

Prompt 3.7 — Validate before answering (CRAG) [KEY PROMPT]

Copy and paste this block to Claude:

```
Implement evaluate_context_relevance(rows, user_query) with gpt-4o
(temperature 0, max_tokens 80). Build a compact snapshot of the top 6
rows and require a strict verdict:
    STATUS: <RELEVANT|IRRELEVANT|AMBIGUOUS>
    REASON: <one sentence, max 25 words>
No rows → IRRELEVANT without an API call. FAIL-OPEN: any error defaults
to RELEVANT so the validator can never block a legitimate answer.
Wire TWO gates into run_query: (1) on the fallback path, validate
BEFORE writing the answer — only RELEVANT proceeds, otherwise return the
refusal template; (2) on the primary path, validate AFTER as
confirmation — a bad verdict downgrades the answer to the refusal.
Surface crag_status and crag_reason in the result.
```

Check: off-topic context produces a refusal, not an invented answer; both verdicts appear in the audit trail.

Prompt 3.8 — Show only the evidence used

Copy and paste this block to Claude:

```
After the answer is written, extract the node IDs and section references
it actually cites, resolve them in the graph, and pass only those to
fetch_reasoning_subgraph for the visual Reasoning View. The picture must
show the grounding for THIS answer, never the whole graph.
```

Check: the rendered subgraph contains only cited nodes, not all 373.

5. Phase 4 — Production & Resilience Features

Goal: make the application run reliably in real environments — Windows consoles, university proxies, free-tier cloud databases — and reproducible from a clean install.

Prompt 4.1 — Fix Windows text encoding

Copy and paste this block to Claude:

```
At the very top of app.py, before any other import, reconfigure stdout
to UTF-8 (sys.stdout.reconfigure(encoding="utf-8") when available) so
log symbols never crash a Windows terminal.
```

Check: the process log prints ✓ and emoji without `UnicodeEncodeError`.

Prompt 4.2 — Trust the university proxy's certificates

Copy and paste this block to Claude:

```
Immediately after, inject the operating system trust store with the
truststore package and build one explicit httpx client carrying that
SSL context (falling back to None if anything fails). This lets HTTPS
calls succeed behind a TLS-intercepting institutional proxy.
```

Check: API and database calls work behind the proxy; without a proxy nothing breaks.

Prompt 4.3 — Use that client everywhere

Copy and paste this block to Claude:

```
Implement _llm_kwargs() returning {"http_client": client} when available,
else {}. Spread **_llm_kwargs() into every ChatOpenAI and embedder
constructor in the codebase, without exception.
```

Check: every model constructor carries the kwargs; removing the proxy still works.

Prompt 4.4 — Survive an embedding outage

Copy and paste this block to Claude:

```
In _init_embedder, after building the OpenAI embedder, run a live probe
(embed_query("probe")). If it fails, fall back to the local
all-MiniLM-L6-v2 model (384 dimensions) and set EMBEDDING_DIMS to match
so the vector indices are created at the right size. Add
_embed_with_retry(max_attempts=3) for transient network errors.
```

Check: with a working key, dimensions are 1536; with OpenAI blocked, the app degrades to 384-dim local embeddings and still works.

Prompt 4.5 — Self-healing database connections

Copy and paste this block to Claude:

```
init_neo4j() and init_chain() are cached. On failure they must clear
their own cache so the next page refresh retries (AuraDB free tier
sleeps and needs a cold start). Add a sidebar "Reconnect" button that
clears both caches on demand. Refresh the live schema after connecting
so query generation always sees the real graph.
```

Check: a sleeping database that fails once recovers on the next refresh or via Reconnect — no restart needed.

Prompt 4.6 — Close the packaging

Copy and paste this block to Claude:

```
Ensure Requirements.txt lists every import (add streamlit-agraph,
altair, pandas, httpx, truststore). Remove all Anthropic references –
```

```
the project is strictly OpenAI — and gate ingestion on OPENAI_API_KEY
only. Confirm python -m py_compile passes and a fresh install boots
the app.
```

Check: a clean virtual environment installs and runs the app; no dead imports remain.

6. Final Integration

Prompt F.1 — Assemble everything into app.py

Copy and paste this block to Claude:

```
Integrate all components from Phases 1-4 into ONE Streamlit file, app.py,
in this order: (1) the UTF-8 and SSL fixes at the very top; (2) config
and constants; (3) infrastructure (embedder with fallback, cached
self-healing connections); (4) ingestion (PDF→XML→Neo4j with labels,
risk flags, indices, list/delete); (5) the query pipeline (clean →
route → structured-first → hybrid fallback → CRAG gates → grounded
answer → reasoning subgraph); (6) the UI: a sidebar with connection
status, policy list and Reconnect; an ingestion panel with a live log;
a Q&A panel showing the answer, generated Cypher, route, CRAG verdict
and per-node scores; and a Reasoning View tab with risk nodes in red.
No dead code; every model constructor spreads **_llm_kwargs().
Finish with a compile check and a headless boot test.
```

Check: `app.py` compiles, launches, ingests a PDF end-to-end, and answers a question with citation, route, CRAG verdict and a reasoning subgraph of only the cited nodes.

7. Code Quality, Documentation & Maintainability

The system was built to be **read, audited and extended**, not just to run. Five practices carry this:

1. Self-documenting structure. The application is one deliberately ordered file (`app.py`) whose layout mirrors the data flow: environment fixes → configuration → infrastructure → ingestion → query pipeline → UI. A reader can follow a question through the file top-to-bottom. Constants that govern behaviour (`CHUNK_SIZE`, `SEMANTIC_THRESHOLD`, `RISK_KEYWORDS`) are named, module-level and grouped, never buried as magic numbers.

2. Docstrings that record *why*, not just *what*. Design rationale is written where the decision lives — e.g. the chunking docstring explains *why* 15,000-character windows with 1,500-character overlap produce more reliable XML, and the search engine documents *why* the cosine threshold is deliberately not applied in direct-lookup mode.

3. Fail-loud error discipline. No error is swallowed. Extraction failures, malformed XML, and database faults are logged with their cause and re-raised to the interface, so a problem can never masquerade as "no data". Conversely, components whose failure should not block users (the router, the CRAG validator) follow an explicit, documented *fail-open* contract.

4. Idempotency and reproducibility. All graph writes use `MERGE` keyed on section-anchored IDs, so re-running ingestion is safe. Model temperatures are 0 wherever determinism matters, making evaluation runs reproducible. `Requirements.txt` declares every dependency, and the repository's `CLAUDE.md` documents the full environment setup and cross-machine workflow.

5. Cost-aware engineering. Expensive calls are cached (`@st.cache_data` on label generation; `@st.cache_resource` on connections, with self-healing invalidation), and cheap models (gpt-4o-mini) are used where a frontier model adds nothing — a documented two-tier cost strategy.

Together these mean a new maintainer — or an examiner — can trace any answer the system gives back to a specific function, a specific design decision, and a specific policy section.

8. Comparison with LightRAG

LightRAG (Guo et al., 2024) is a popular lightweight graph-based RAG framework developed by researchers at the University of Hong Kong and Beijing University of Posts and Telecommunications. While it offers an efficient approach to graph-enhanced retrieval, our system differs significantly in design priorities for regulatory and institutional use cases.

Realistic Example:

Question: *"Can my supervisor also act as my internal examiner?"*

- **LightRAG (Guo et al., 2024)**: Retrieves relevant text chunks containing “supervisor” and “examiner” and generates a fluent answer. However, it may not explicitly detect contradictions between clauses and typically lacks precise clause-level provenance.
- **Transparent Policy GraphRAG**: Routes the question to a structured Cypher query, traverses the HAS_CONDITION and CONFLICTS_WITH relationships, identifies the exact conflicting rules (C_5.1.2 and C_5.1.6), and presents both the answer and visual reasoning subgraph with direct section citations.

This demonstrates our system’s strength in **deterministic conflict detection** and **auditability** at the cost of requiring an upfront schema design.

Aspect	LightRAG	Transparent Policy GraphRAG
Graph construction	LLM freely extracts entities and relations from text	Schema-constrained: only Policy, Rule, Condition, Outcome with typed edges
Node identity	Generated entity names; provenance approximate	Section-anchored IDs (`C_13_1_LawThesisWordLimit`) — every node cites its source clause
Retrieval	Dual-level keyword + graph lookup	Adaptive routing → structured Cypher first → hybrid vector + keyword fusion
Answer validation	None built in	CRAG gate: refuses rather than guesses when evidence is irrelevant
Auditability	Limited — graph is an internal aid	First-class: generated Cypher, scores, verdicts and a reasoning subgraph are shown

A small illustrative example. Ask both systems: *"What happens if a student exceeds the thesis word limit?"*

- **LightRAG** might retrieve entities such as *thesis*, *word limit*, *penalty* and compose a fluent answer — but if the corpus also mentions word limits in an unrelated context (say, an abstract limit), nothing structurally prevents the two from being blended, and the answer carries no clause-level citation.
- **TP-GraphRAG** resolves the question to the node `C_13_1_LawThesisWordLimit`, traverses its rule's `HAS_OUTCOME` edge to the specific outcome node, and answers with the section number attached. The Reasoning View then displays exactly those three nodes. If the question were instead about something the policy does not cover, the CRAG gate returns a refusal rather than a plausible fabrication.

The trade-off is honest: LightRAG generalises to any document type with no schema design effort, while TP-GraphRAG accepts an upfront ontology cost in exchange for the verifiability that governance and compliance use cases require.

9. Research Comparison Table

Dimension	Vector RAG (baseline)	Microsoft GraphRAG	LightRAG	TP-GraphRAG (this work)
Knowledge representation	Text chunks + embeddings	LLM-extracted entity graph + community summaries	Lightweight entity–relation graph + key-value store	Schema-constrained policy ontology in Neo4j
Provenance granularity	Chunk-level	Community / entity level	Entity level	**Clause level** (section-anchored IDs)
Retrieval strategy	Top-k similarity	Global (community) + local search	Dual-level (low/high) keyword-graph	Adaptive routing → Cypher-first → RRF-fused hybrid fallback
Hallucination control	None inherent	Summaries reduce but don't gate	None inherent	**CRAG validation gate with explicit refusal**
Indexing cost	Low	High (many LLM calls for summarisation)	Moderate	Moderate (one extraction pass + cheap labelling)
Transparency to end user	Low	Moderate	Moderate	**High** (Cypher, scores, verdicts, reasoning subgraph)
Best suited to	General Q&A at scale	Corpus-wide thematic questions	Fast general-purpose graph RAG	**Auditable institutional governance & compliance**

The positioning is deliberate: TP-GraphRAG does not aim to outperform general-purpose frameworks on open-domain breadth. It targets the dimension the others leave weakest — **verifiable, clause-level, refusal-capable answering** — which is precisely what policy and compliance auditing demands.

10. Conclusion

Following this guide produces a complete, working system: a policy PDF becomes a validated knowledge graph; questions are routed intelligently, answered structurally where possible, and validated before any text is generated; and every answer arrives with its evidence, its scores, and a visual reasoning trace. Compared with standard RAG — which retrieves loose text by similarity and trusts the language model to compose honestly — this approach makes the system's reasoning **transparent, reproducible and auditable**, and gives it the institutional virtue most generative systems lack: the ability to say *"the policy does not cover this"* instead of inventing an answer.

Repository: github.com/AllenSharafzad/kg-bot-project (branch: master)

References:

Cormack, G.V., Clarke, C.L.A. and Buettcher, S. (2009) 'Reciprocal rank fusion outperforms condorcet and individual rank learning methods', Proceedings of SIGIR '09, ACM, pp. 758–759.

Edge, D. et al. (2024) 'From Local to Global: A Graph RAG Approach to Query-Focused Summarization'. arXiv:2404.16130.

Guo, Z. et al. (2024) 'LightRAG: Simple and Fast Retrieval-Augmented Generation'. arXiv:2410.05779.

Lewis, P. et al. (2020) 'Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks', Advances in Neural Information Processing Systems, 33. arXiv:2005.11401.

Yan, S.-Q. et al. (2024) 'Corrective Retrieval Augmented Generation'. arXiv:2401.15884.